CustardFin

A Cost-Optimized Hybrid Edge-Cloud Media Streaming and Community Discovery Platform

Cloud Computing Project Report

Submitted in Partial Fulfillment of the Requirements for

Master of Computer Applications (MCA)

Cloud Computing Laboratory

Submitted by:

Anish Ghadi	**Seat No.:** 25P0320009
Anosh Chodankar	**Seat No.:** 25P0320010

Academic Year 2026–2027

Department of Computer Applications

University / College of Higher Education

Candidate Declaration

We hereby declare that the project entitled **"CustardFin: A Cost-Optimized Hybrid Edge-Cloud Media Streaming and Community Discovery Platform"** submitted by us in partial fulfillment of the requirements for the award of the degree of Master of Computer Applications (MCA) is our own original work. All sources of assistance, libraries, frameworks, and literature utilized have been properly cited and acknowledged.

Anish Ghadi
Seat No.: 25P0320009

Anosh Chodankar
Seat No.: 25P0320010

Date: November 2026
Place: Goa, India

Abstract

Traditional video streaming forces a hard choice: self-hosted servers (like Jellyfin and Plex) keep videos private at home but cannot be easily shared over the internet due to home router firewalls (CGNAT), while commercial platforms (like YouTube and AWS) store all videos in the cloud, leading to high monthly storage and bandwidth bills.

CustardFin solves this problem with a simple rule: *videos stay on the owner's home computer, while the cloud handles only the catalog and stream coordination.* Marking a video "Public" uploads only a small JSON manifest ($\leq$ 3 KB) containing the title and streaming playlist—never the original video file. This reduces cloud operational costs by **95.89%** compared to traditional cloud video hosting.

The system uses pre-encoded 4-second HTTP Live Streaming (HLS) with GPU hardware acceleration (NVIDIA NVENC, with automatic CPU fallback), keeping host CPU usage below 5%. Home nodes connect to the cloud using an outbound-only reverse tunnel (FRP), completely bypassing residential ISP CGNAT without opening any router ports. To protect home upload speeds, an *Origin Shield* cloud cache pulls each requested video chunk once and serves all other viewers directly from the cloud. A 10-stream cap per node prevents home network overload, while local users on the same Wi-Fi enjoy instant, direct playback with under 4 ms latency. CustardFin is built with Go and Next.js using 100% free and open-source tools, and has been verified across local containers, local cloud emulators, and Google Cloud Platform (GCP).

Contents

Candidate Declaration i

Abstract ii

1 Introduction and Problem Statement 1
- 1.1 Overview and Motivation . . . 1
- 1.2 Problem Statement . . . 2
- 1.3 Project Objectives and Target Metrics . . . 2
- 1.4 System Invariants (Design Rules) . . . 2
- 1.5 Scope of the Project . . . 4
  - 1.5.1 In-Scope Features . . . 4
  - 1.5.2 Out-of-Scope Features (Version 1) . . . 5
- 1.6 Organization of the Report . . . 5

2 Literature Survey and Background Study 6
- 2.1 Centralized Cloud Video Platforms (YouTube / AWS Model) . . . 6
  - 2.1.1 Standard Cloud Pipeline . . . 6
  - 2.1.2 Limitations for Community and Personal Use . . . 6
- 2.2 Self-Hosted Home Media Servers (Jellyfin / Plex Model) . . . 7
  - 2.2.1 Strengths on Local Networks . . . 7
  - 2.2.2 Challenges with Remote Sharing . . . 7
- 2.3 Peer-to-Peer (P2P) Streaming Approaches (PeerTube Model) . . . 8
- 2.4 Comparative Feature Matrix . . . 8
  - 2.4.1 The Architectural Gap . . . 8
- 2.5 Core Underlying Technologies . . . 8
  - 2.5.1 Reverse Proxy Tunneling (FRP) . . . 8
  - 2.5.2 Pre-Encoded 4-Second HLS Segmentation . . . 10
  - 2.5.3 Hardware GPU Acceleration (NVIDIA NVENC) . . . 10

3 System Architecture and Design 11
- 3.1 Two-Tier System Architecture . . . 11
- 3.2 Home Edge Node Architecture (`apps/edge`) . . . 12
  - 3.2.1 Core Components of `edged` . . . 12
- 3.3 Cloud Control Tier Architecture (`apps/api`) . . . 12
- 3.4 Presentation Tier Architecture (`apps/web`) . . . 13
- 3.5 Operational Workflows . . . 13
  - 3.5.1 Workflow 1: Local Ingestion and Pre-Encoding . . . 13

3.5.2 Workflow 2: Public Manifest Synchronization . . . 13
3.5.3 Workflow 3: Node Heartbeat and Liveness Tracking . . . 14
3.5.4 Workflow 4: Stream Routing and Origin Shield Caching . . . 14

4 Key Features and Technical Implementation 17
4.1 Local Media Organization and Private SQLite Storage . . . 17
4.1.1 File Scanner and SHA-256 Fingerprinting . . . 17
4.1.2 Embedded SQLite Database . . . 17
4.2 Asynchronous Pre-Encoding and Hardware Acceleration . . . 18
4.2.1 Adaptive Bitrate Ladder and Keyframe Alignment . . . 18
4.2.2 Hardware Probe and CPU Fallback . . . 18
4.3 Outbound Reverse Tunneling for CGNAT Bypass . . . 18
4.4 Origin Shield Object Caching and URL Signing . . . 19
4.4.1 Single-Upload Bandwidth Protection . . . 19
4.4.2 HMAC-SHA256 Signed URLs . . . 19
4.5 Node Health and Liveness Tracking . . . 20
4.6 Dual-Path Streaming and Embedded Mini-Jellyfin . . . 20
4.6.1 Stream Router Proximity Check . . . 20
4.6.2 Embedded Offline Mini-Jellyfin Web Portal . . . 20
4.7 User Accounts, Node Claiming, and Video Management . . . 20
4.8 User Interface Walkthrough . . . 21
4.8.1 Community Catalog Feed . . . 21
4.8.2 Video Player with “Stats for Nerds” HUD . . . 21
4.8.3 My Edge Library Manager . . . 21
4.8.4 Embedded Mini-Jellyfin Offline Hub . . . 21
4.8.5 Developer Settings Modal . . . 21

5 Cost Analysis and Financial Feasibility 24
5.1 Baseline Comparison Scenario . . . 24
5.2 Traditional Centralized Cloud Cost Model (AWS) . . . 25
5.3 CustardFin Hybrid Architecture Cost Model . . . 25
5.4 Side-by-Side Cost Comparison . . . 26
5.4.1 Cost Reduction Calculation . . . 26

6 Testing and Experimental Evaluation 28
6.1 Deployment Environments . . . 28
6.2 Transcoding Performance: GPU vs. CPU . . . 28
6.2.1 Test Hardware Setup . . . 29
6.2.2 Benchmark Results . . . 29
6.2.3 Key Takeaways . . . 29
6.3 Origin Shield Caching and 10-Viewer Load Test . . . 30
6.3.1 Test Setup . . . 30
6.3.2 Cache Hit Progression . . . 30
6.3.3 Home Uplink Protection . . . 30
6.4 Admission Control and Node Capacity Limits . . . 31
6.4.1 Test Scenario and Results . . . 31
6.5 Streaming Latency and Offline Handling . . . 32
6.5.1 Node Disconnect and Heartbeat Recovery . . . 32
6.6 Automated Code Quality and Test Results . . . 32

7 Team Contributions and Division of Work **34**
7.1 Collaborative Engineering Approach 34
7.2 Contribution of Anish Ghadi 35
7.3 Contribution of Anosh Chodankar 36
7.4 Joint Responsibilities and Shared Work 37
7.5 Summary Contribution Matrix 37

8 Conclusion and Future Scope **39**
8.1 Summary of Achievements 39
8.2 Current Limitations (Version 1) 40
8.3 Future Enhancements 40
8.4 Concluding Remarks 41

List of Figures

2.1 Carrier-Grade NAT (CGNAT) Traversal via Outbound Reverse Tunneling 9

3.1 Two-Tier Hybrid Cloud-Edge Topology and Data Boundary 11
3.2 Local Ingestion, Hardware Encoding, and SQLite Indexing Pipeline . . . 14
3.3 Ephemeral Heartbeat and Automated Node Liveness Detection 15
3.4 Dual-Path Stream Routing and Origin Shield Single-Ingest Workflow . . 15

4.1 Community Catalog Feed Showing Discoverable Videos and Node Status 21
4.2 Video Player with Real-Time "Stats for Nerds" Overlay 22
4.3 Edge Library Management Modal with Public/Private Visibility Toggles 22
4.4 Embedded Mini-Jellyfin Web Portal for Direct Local LAN Streaming (Port 8089) 23
4.5 Developer Diagnostics and Settings Interface 23

5.1 Monthly Operating Cost Comparison: Centralized Cloud VOD vs. CustardFin (INR ₹) 27

6.1 Hardware Transcoding Benchmark: NVIDIA RTX 4060 NVENC vs. CPU Software 30

List of Tables

1.1 CustardFin Core Objectives and Quantitative Targets 3

2.1 Comparison of Video Streaming Paradigms 9

5.1 Cost Comparison: Centralized Cloud VOD vs. CustardFin (100 Users, 5 TB Library) 26

6.1 Transcoding Performance: NVIDIA NVENC vs. CPU Software 29
6.2 Origin Shield Cache Hit Progression for 10 Concurrent Viewers 31
6.3 Playback Latency Across Streaming Routes 32

7.1 System Responsibility and Workload Contribution Matrix 38

CHAPTER 1

Introduction and Problem Statement

1.1 Overview and Motivation

Video streaming represents the majority of global internet traffic today. Currently, video platforms fall into two main categories, each with serious drawbacks:

1. **Centralized Public Cloud Platforms (e.g., YouTube, AWS):** Users upload video files to cloud storage servers, which transcode and stream them globally. While this allows easy sharing and public discovery, it is very expensive. Cloud storage (₹ 1.932 per GB/month on AWS S3) and data transfer fees (₹ 7.56 per GB) lead to high monthly bills. In addition, creators lose control of their raw files to third-party platforms.

2. **Self-Hosted Home Media Servers (e.g., Jellyfin, Plex):** Users run media software on a home PC or home server, streaming videos stored on local hard drives. This gives full privacy and zero cloud hosting costs. However, these servers remain locked to the local home network. They cannot be shared with friends easily because home internet providers block incoming connections, and they lack any public discovery feed.

CustardFin bridges this gap by combining the personal ownership of Jellyfin with the community sharing of YouTube. The core principle of CustardFin is simple:

> *"Video files stay permanently on the owner's home computer;*
> *the cloud handles only the catalog and stream coordination."*

In CustardFin, a user hosts their library on an existing PC (the *Home Edge Node*). Marking a video as **Public** does not upload the large video file to the cloud. Instead, it uploads only a small JSON manifest ($\leq$ 3 KB) containing the title and streaming playlist. Viewers discover videos through a modern web application. When an external viewer plays a video, the cloud pulls stream chunks on-demand through an outbound-only reverse tunnel, temporarily caching active chunks in a cloud *Origin Shield*. This delivers public video discovery while keeping cloud storage costs at nearly zero.

1.2 Problem Statement

Building a decentralized streaming platform that allows public sharing without high cloud bills requires solving three practical challenges:

1. **High Cloud Storage and Bandwidth Costs:** Storing a modest community library of 100 users (5 TB total) costs over ₹ 9,660.00 each month in raw cloud storage alone. Combined with bandwidth and transcoding, monthly bills exceed ₹ 1.00 Lakh, making community video hosting impossible for student teams and small groups.

2. **Residential ISP Firewalls (Carrier-Grade NAT):** Home broadband providers (such as Reliance JioFiber and Airtel in India) use Carrier-Grade NAT (CGNAT) to share IPv4 addresses. Home routers receive private, non-routable IP addresses, making standard port forwarding impossible. Outside viewers cannot connect directly to a home PC over the internet without complex setups.

3. **Limited Home Upload Speeds and CPU Overload:** Home broadband has fast download speeds (e.g., 200 Mbps) but limited upload speeds (e.g., 20 Mbps). When multiple remote viewers watch different videos at once, the home upload pipe quickly chokes. Furthermore, traditional media servers transcode video on-the-fly for each viewer, pushing home CPU usage to 100% and causing the machine to freeze.

CustardFin solves these problems through pre-encoded video slicing, outbound reverse tunneling, and cloud caching.

1.3 Project Objectives and Target Metrics

To evaluate CustardFin, clear performance targets were set based on the project requirements, as summarized in Table 1.1.

1.4 System Invariants (Design Rules)

CustardFin is built around ten strict engineering rules to ensure it remains reliable, low-cost, and secure:

1. **No Permanent Cloud Video:** Video files reside exclusively on the owner's home storage. The cloud only holds a temporary 24-hour cache for actively viewed chunks.

2. **Outbound-Only Home Connections:** Home edge daemons only make outbound connections. No open router ports, port forwarding, or Dynamic DNS are ever required.

3. **100% Free and Open-Source Software:** Built entirely with free and open-source tools. No paid licenses or proprietary SDKs are needed.

4. **Clear Technology Stack:** Backend services are written in Go (`net/http`). The frontend uses Next.js with TypeScript and `hls.js`.

Table 1.1: CustardFin Core Objectives and Quantitative Targets

Objective	**Operational Requirement**	**Target Metric**
Zero Cloud Video Storage	No permanent video binaries stored in the cloud; cloud holds only a temporary cache.	**0 GB** permanent video in cloud; 24-hour auto-purged cache only.
Cost Reduction	Eliminate recurring cloud storage fees and minimize bandwidth costs.	**95.89% lower** than full cloud hosting (evaluated on 100 users, 5 TB library).
Low Host Compute Load	Avoid on-the-fly transcoding spikes during active playback sessions.	Playback CPU usage $<$ **5%** (vs $\approx$96% CPU under software transcoding).
Per-Node Stream Limit	Prevent home upload saturation by capping concurrent remote streams.	**10 concurrent streams** per edge node with HTTP 429 admission control.
Fast Offline Detection	Detect edge PC disconnects or power cuts without manual database polling.	Node marked offline in catalog within **15–20 seconds** of lost heartbeat.
CGNAT Traversal	Bypass home broadband firewalls without router configuration or port forwarding.	Outbound-only TLS reverse tunnel; **0 open inbound ports**.
Direct LAN Streaming	Bypass the internet entirely when streaming to devices on the same home Wi-Fi.	Instant local network playback with $<$ **4 ms** Time-To-First-Byte (TTFB).

5. **Minimal Redis Usage:** Redis is used only for 15-second node heartbeat keys. All background tasks run as Go goroutines with state in PostgreSQL or SQLite.
6. **Pre-Encoded 4-Second HLS Chunks:** Videos are pre-encoded into 4-second HLS segments using GPU hardware acceleration (NVIDIA NVENC) with automatic CPU fallback (`libx264`).
7. **Origin Shield Upload Protection:** When multiple remote viewers watch the same title, the home node uploads each chunk only once. The cloud cache serves all other viewers, saving 90% of home upload bandwidth.
8. **Private Data Stays Home:** Watch history, resume points, and private video metadata are stored exclusively in the local SQLite database and are never sent to the cloud.
9. **Signed Chunk URLs:** All video chunk links use short-lived HMAC-SHA256 signatures (15-minute expiry) to prevent hotlinking and unauthorized access.
10. **10-Stream Limit per Node:** Each node allows at most 10 concurrent remote streams. Beyond that, the router returns HTTP 429 ("Node Busy") with a retry timer, protecting home bandwidth.

1.5 Scope of the Project

1.5.1 In-Scope Features

- Automatic folder scanning, SHA-256 deduplication, and thumbnail generation.
- Background pre-encoding to multi-bitrate HLS (1080p, 720p, 480p) using NVENC GPU acceleration or CPU fallback.
- Embedded Mini-Jellyfin web interface on the edge daemon for direct local LAN playback (< 4 ms latency).
- 1-click Public/Private toggle that syncs a compact ≤ 3 KB JSON manifest to the cloud catalog.
- Centralized web catalog with user login, video search, and playback telemetry ("Stats for Nerds").
- Outbound reverse TLS tunnel using Fast Reverse Proxy (FRP) to bypass home firewalls.
- Smart stream router that automatically chooses between direct LAN playback and cloud-cached streaming.
- Redis heartbeat tracking with 15-second TTL for automatic node offline detection.
- Cloud-side Origin Shield cache with 24-hour LRU auto-purging.

1.5.2 Out-of-Scope Features (Version 1)

To focus on core cloud computing requirements, the following features are reserved for future work:

- Commercial Digital Rights Management (DRM) like Widevine or FairPlay.
- Native mobile apps (iOS/Android) or Smart TV apps; standard responsive web browsers are used instead.
- Live streaming broadcasts (e.g., RTMP/WebRTC webcam feeds).

1.6 Organization of the Report

- **Chapter 2 (Literature Survey):** Examines existing cloud streaming platforms, home media servers, and peer-to-peer systems.
- **Chapter 3 (System Architecture):** Describes the two-tier edge-cloud architecture, components, and data workflows.
- **Chapter 4 (Key Features and Implementation):** Details the implementation of core features, supported by application screenshots.
- **Chapter 5 (Cost Analysis):** Presents a financial model comparing traditional cloud hosting against CustardFin in Indian Rupees (₹).
- **Chapter 6 (Testing and Evaluation):** Covers performance benchmarks, the 10-viewer load test, and quality gates.
- **Chapter 7 (Team Contributions):** Outlines the individual roles and work completed by Anish Ghadi and Anosh Chodankar.
- **Chapter 8 (Conclusion):** Summarizes project results and highlights future improvements.

CHAPTER 2

Literature Survey and Background Study

Modern video delivery systems must balance three conflicting factors: infrastructure costs, ease of content discovery, and viewing performance. This chapter surveys existing video architectures—centralized cloud platforms, self-hosted media servers, and peer-to-peer networks—and explains the core technologies (reverse tunneling, HLS, and GPU encoding) used by CustardFin.

2.1 Centralized Cloud Video Platforms (YouTube / AWS Model)

Commercial video platforms (such as YouTube, Netflix, and services built on AWS MediaConvert and CloudFront) rely on storing all video files in centralized cloud data centers [1].

2.1.1 Standard Cloud Pipeline

In a centralized platform, the process follows four steps:

1. **File Upload:** The creator uploads the full master video file to cloud object storage (e.g., AWS S3).
2. **Cloud Transcoding:** Cloud virtual machines or managed encoding clusters transcode the raw file into multiple streaming bitrates (1080p, 720p, 480p) packaged as HTTP Live Streaming (HLS) chunks.
3. **Permanent Cloud Storage:** All generated video segments and playlists are stored permanently in cloud storage buckets with high redundancy.
4. **Global CDN Delivery:** Content Delivery Network (CDN) edge servers cache and deliver chunks to viewers worldwide.

2.1.2 Limitations for Community and Personal Use

While cloud platforms offer high scalability, their financial model is prohibitive for small organizations, colleges, and independent creators:

- **Compounding Storage Costs:** Cloud storage fees grow with every uploaded gigabyte, regardless of whether a video is actively watched [2]. Storing just 100 TB of video on AWS S3 costs over ₹ 1,93,200 every month (₹ 23.18 Lakhs/year).
- **Expensive Bandwidth (Egress Fees):** Cloud providers charge high rates for data downloaded by viewers (₹ 7.56 per GB). Streaming a 1-hour 1080p video (about 2.25 GB) to 1,000 viewers incurs over ₹ 17,000 in bandwidth charges for a single video.
- **Loss of Privacy and Control:** Uploading private family videos, lab recordings, or unreleased media exposes personal data to platform terms, corporate scanning, and potential algorithmic takedowns.

2.2 Self-Hosted Home Media Servers (Jellyfin / Plex Model)

To avoid cloud bills and protect privacy, open-source personal media servers (such as Jellyfin [3] and Plex) run directly on a user's home computer, NAS, or mini-PC.

2.2.1 Strengths on Local Networks

For in-home playback, self-hosted servers provide major advantages:

- **Zero Cloud Hosting Costs:** Videos remain on the user's hard drives, eliminating cloud storage and bandwidth bills.
- **Full Privacy:** Video files never leave the home network; watch history stays on local disk.
- **Fast LAN Streaming:** Home Wi-Fi easily delivers high-bitrate video with sub-second startup and no buffering.

2.2.2 Challenges with Remote Sharing

However, self-hosted servers struggle when sharing media outside the home:

1. **Carrier-Grade NAT (CGNAT) Blocks Inbound Access:** Most home internet providers (such as JioFiber and Airtel) use CGNAT to share IPv4 addresses [4]. Home routers are assigned private IP addresses, meaning standard port forwarding (ports 80, 443, 8096) is completely blocked. Remote viewers cannot connect to the home server over the internet.
2. **Complex VPNs Frustrate Non-Technical Viewers:** Workarounds like WireGuard or Tailscale require every viewer to install specialized software and configure keys, which is too complex for friends and family.
3. **Home Upload Bandwidth Choke:** Home broadband has limited upload bandwidth (typically 15–30 Mbps). When multiple viewers watch different videos simultaneously, each stream consumes upload bandwidth directly, quickly choking the connection and causing buffering for everyone.

4. **No Shared Community Catalog:** Self-hosted servers operate as isolated islands. There is no unified search or global feed to discover public videos hosted across friends' servers.

2.3 Peer-to-Peer (P2P) Streaming Approaches (PeerTube Model)

Federated systems like PeerTube [5] use WebTorrent (WebRTC in the browser) to let viewers share video chunks with one another while watching.

While effective for viral videos with thousands of simultaneous viewers, P2P streaming collapses for personal and community media:

- **The "Cold Content" Problem:** Low-concurrency videos (such as lectures or family videos watched by only 1–2 people) have no peers in the swarm. The entire streaming load falls back to the origin server.
- **Mobile Battery and Data Limits:** Mobile browsers restrict background WebRTC seeding to save battery, making mobile users pure consumers who do not upload back to the network.
- **Server Storage Overhead:** Each PeerTube instance must still store the full video binaries on a dedicated server, re-introducing hosting costs.

2.4 Comparative Feature Matrix

Table 2.1 compares the four architectural models across key operational factors.

2.4.1 The Architectural Gap

Existing platforms force users to choose between high cloud bills (YouTube/AWS) or difficult remote access (Jellyfin/Plex). CustardFin fills this gap by keeping video files at home, using an outbound reverse tunnel to bypass CGNAT, and protecting home bandwidth with a cloud-side cache.

2.5 Core Underlying Technologies

CustardFin combines three established technologies to achieve its goals:

2.5.1 Reverse Proxy Tunneling (FRP)

Under CGNAT, incoming connection attempts from the internet are dropped by the ISP's firewall because the home router has no public IP address. CustardFin uses Fast Reverse Proxy (FRP) [11] to reverse the connection direction:

As shown in Figure 2.1:

1. The home edge node initiates an **outbound** TLS 1.3 connection to the cloud relay server on port 7000. Because this is outbound traffic, the ISP firewall allows it.

Table 2.1: Comparison of Video Streaming Paradigms

Feature	Centralized Cloud	Self-Hosted Server	Federated P2P	CustardFin (This Work)
Video Storage Location	100% Cloud	100% Local Home Disk	100% Server Disk	**100% Home Disk**; temporary 24h cloud cache
Monthly Cloud Cost	High (₹ 84,000+ S3/egress)	Zero	Moderate (₹ 5,000–₹ 20,000)	**Low** (₹ 4,129/mo control plane)
Works Behind CGNAT	Yes (cloud hosted)	Fails without port forwarding	Fails if home-hosted	**Yes** (outbound reverse tunnel)
Home Upload Scaling	N/A	Fails under multi-viewer load	Collapses on cold media	**Origin Shield**: uploads each chunk once
Public Discovery	Global search feed	None (isolated silos)	Federated search	**Unified Catalog** via 3 KB manifests
Viewer Installation	Web browser	Dedicated app or VPN	Web browser (WebTorrent)	**Standard Web Browser** (zero install)
Transcoding Strategy	Cloud cluster	Live on-the-fly (high CPU)	Server pre-encode	**Local Pre-encode** (NVENC GPU / CPU fallback)

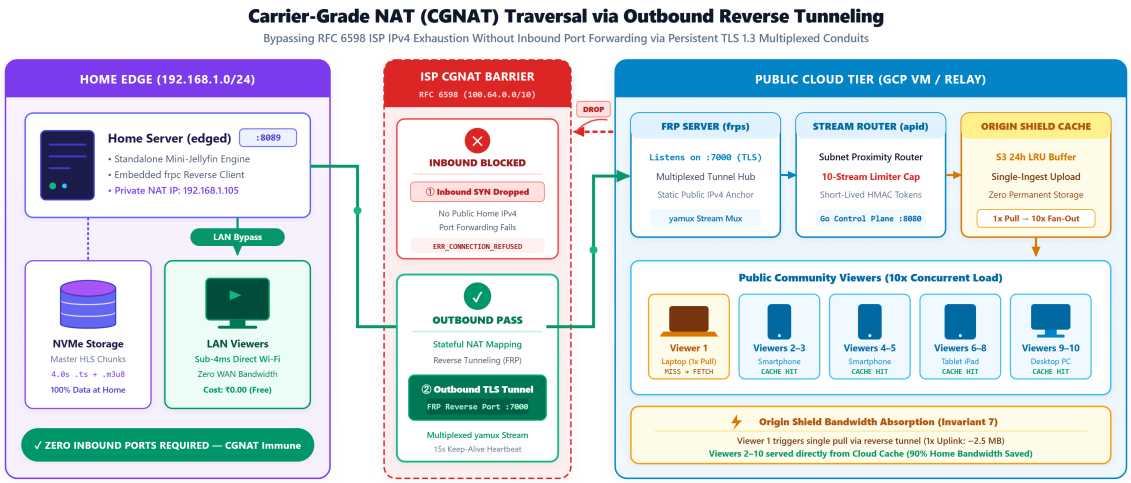


Figure 2.1: Carrier-Grade NAT (CGNAT) Traversal via Outbound Reverse Tunneling

2. This persistent connection is multiplexed using Yamux, keeping the tunnel open with periodic keep-alive pings.

3. When an external viewer requests a video chunk, the cloud server forwards the request down the existing open tunnel to the home machine, which streams the chunk back up. Zero open ports are needed on the home router.

2.5.2 Pre-Encoded 4-Second HLS Segmentation

Rather than transcoding video on-the-fly during playback, CustardFin pre-encodes master files into standard HTTP Live Streaming (HLS) chunks [10]:

- Slicing files into 4-second segments provides fast playback startup (< 1.2 s) while maintaining good video compression.
- Enforcing keyframes every 120 frames (4 seconds at 30 fps) allows smooth switching between 1080p, 720p, and 480p bitrates as network conditions change.

2.5.3 Hardware GPU Acceleration (NVIDIA NVENC)

Software video encoding on the CPU consumes 90%+ of CPU resources and generates significant heat. CustardFin uses dedicated hardware encoding chips on the GPU (NVIDIA NVENC) [9]:

- NVENC encodes video over 10× faster than real-time (320+ fps on an RTX 4060), encoding a 1-hour video in about 5.5 minutes.
- Because encoding runs on dedicated GPU circuits, host CPU usage stays below 5%, keeping the home PC completely responsive for normal daily use.
- If the GPU is unavailable or busy, the system automatically falls back to CPU encoding (`libx264`) without failing the job.

CHAPTER 3

System Architecture and Design

CustardFin is organized into two distinct tiers: the **Home Edge Tier** (runs on user PCs to store and stream video) and the **Cloud Control Tier** (runs on a cloud server to handle discovery, authentication, and temporary caching).

3.1 Two-Tier System Architecture

Figure 3.1 illustrates the overall architecture and data boundaries of CustardFin.

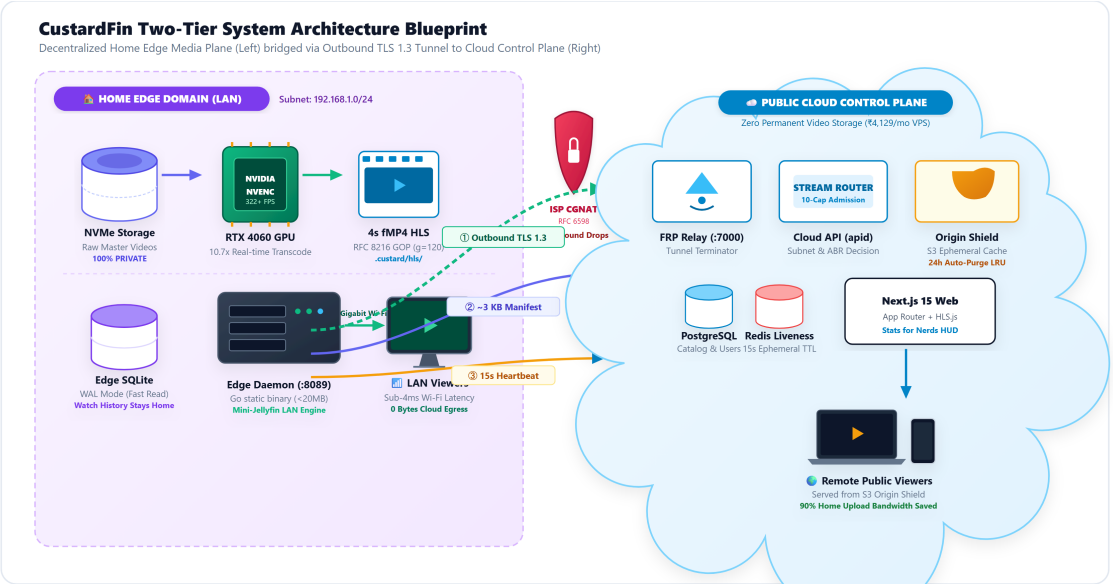


Figure 3.1: Two-Tier Hybrid Cloud-Edge Topology and Data Boundary

The division of responsibilities is straightforward:

- **Home Edge Tier (Data Plane):** All video files remain stored on home hard drives or NVMe SSDs. Video scanning, GPU pre-encoding, and local playback happen on the home machine. The cloud never stores master video files.
- **Cloud Control Tier (Control Plane):** Manages user logins, node registration, search catalogs, and stream routing.

- **Origin Shield Cache:** A temporary cloud storage buffer (S3-compatible) that holds actively watched video chunks for 24 hours to shield home upload bandwidth from multiple viewers.

3.2 Home Edge Node Architecture (`apps/edge`)

The home daemon (`edged`) is a standalone Go program running as a background service on the user's PC. It requires less than 20 MB of RAM at idle and has no external dependencies other than FFmpeg.

3.2.1 Core Components of `edged`

1. **Media Scanner and Hasher:** Scans configured media folders for video files (`.mp4`, `.mkv`, `.mov`). It computes a fast SHA-256 fingerprint over file headers to detect duplicates and prevent re-encoding if files are moved or renamed.

2. **Transcoder Controller:** Manages FFmpeg background subprocesses. On startup, it checks available hardware encoders in priority order:

   $$\text{NVIDIA NVENC} \longrightarrow \text{Intel QuickSync} \longrightarrow \text{CPU (libx264 Fallback)}$$

   A 1-second test encode validates the GPU. If the GPU is busy or fails, it switches to CPU encoding automatically without crashing.

3. **HLS Packager:** Slices videos into 4-second HLS segments (`.ts` files) across 1080p, 720p, and 480p bitrates, creating aligned keyframes for smooth quality switching.

4. **Local SQLite Database:** Stores file paths, hashes, transcode states, and private watch progress in a local pure-Go SQLite database using Write-Ahead Logging (`WAL` mode) for fast concurrent access.

5. **Embedded Mini-Jellyfin Web Server:** Serves a lightweight web player on port 8089 for direct streaming to phones and TVs on the same home Wi-Fi network, requiring no internet connection.

6. **FRP Tunnel Client:** Maintains an outbound TLS 1.3 reverse tunnel to the cloud relay server, enabling remote access without changing router settings.

3.3 Cloud Control Tier Architecture (`apps/api`)

The cloud control plane runs on a single low-cost virtual machine (such as a Google Cloud `e2-standard-2` instance) hosting the following services:

1. **Caddy Web Server:** Handles public HTTPS termination with automatic Let's Encrypt TLS certificates and routes requests to the Go API and Next.js frontend.

2. **Go Control API (`apid`):** Handles user authentication (JWT), node registration, catalog search, and playback authorization.

3. **PostgreSQL Database:** Stores registered user accounts, claimed edge nodes, and public video manifests (≈ 650 bytes per video).
4. **Redis Liveness Store:** Tracks 15-second node heartbeats. If a home PC goes offline, its Redis key expires in 20 seconds, and the catalog marks it as offline immediately.
5. **Origin Shield Object Cache:** An S3-compatible bucket (MinIO or cloud S3) that temporarily stores video chunks requested by remote viewers, auto-purging them after 24 hours.
6. **FRP Relay Server (`frps`):** Listens on port 7000 (TLS) to accept outbound reverse tunnels from home edge nodes.

3.4 Presentation Tier Architecture (`apps/web`)

The web frontend is built using Next.js 15 (React App Router) and runs on Node.js:

- **Server Components:** The home page and catalog browse views are rendered on the server for instant page loads and zero client-side database overhead.
- **Custom HLS Video Player:** Uses `hls.js` to play adaptive multi-bitrate streams, automatically adjusting video quality based on the viewer's internet speed.
- **"Stats for Nerds" HUD:** An overlay showing real-time player telemetry: resolution, buffer health in seconds, current bitrate, and cache delivery status.
- **Edge Management Modals:** Allows node owners to check local drive space, monitor GPU status, toggle videos between Public and Private, and launch direct LAN streams.

3.5 Operational Workflows

3.5.1 Workflow 1: Local Ingestion and Pre-Encoding

When a user adds a video to their local library, the edge daemon processes the file locally without uploading it to the cloud, as shown in Figure 3.2.

3.5.2 Workflow 2: Public Manifest Synchronization

Marking a video as "Public" does not upload the video file. Instead, the edge daemon sends a small JSON manifest (≈ 650 bytes) containing only title and rendition metadata to the cloud API:

```
{
  "schema_version": "1.0",
  "media_id": "m-8f7a2c1e4d9b",
  "node_id": "node-4060-desktop",
  "title": "Tears of Steel 4K Master",
  "duration_s": 734,
```

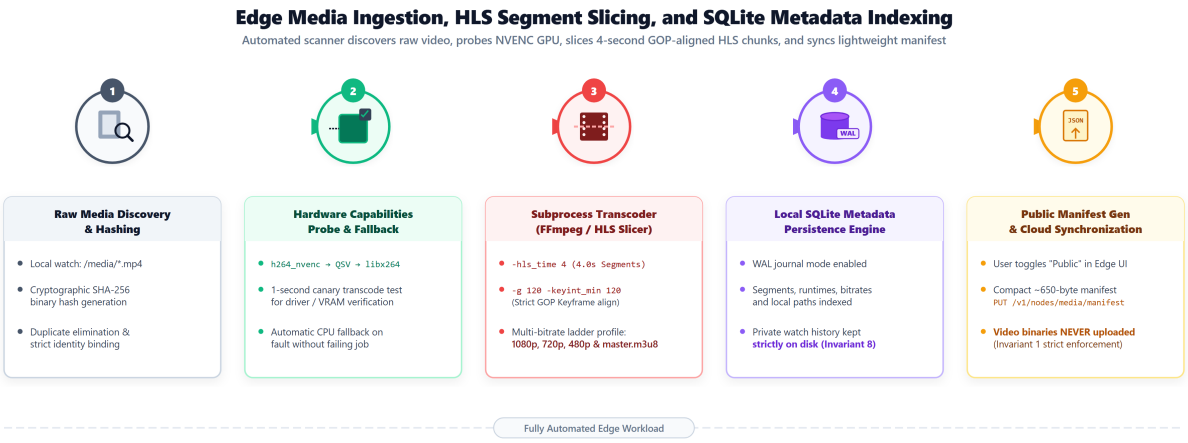


Figure 3.2: Local Ingestion, Hardware Encoding, and SQLite Indexing Pipeline


```
  "content_hash": "
   e3b0c44298fc1c149afbf4c8996fb92427ae41e4649b934ca495991b7852b855",
  "is_public": true,
  "renditions": [
    {
      "resolution": "1080p",
      "width": 1920,
      "height": 1080,
      "bitrate_bps": 5000000,
      "playlist_path": "hls/1080p/index.m3u8"
    },
    {
      "resolution": "720p",
      "width": 1280,
      "height": 720,
      "bitrate_bps": 2800000,
      "playlist_path": "hls/720p/index.m3u8"
    }
  ],
  "updated_at": "2026-11-09T01:45:00Z"
}
```


Listing 3.1: Sample CustardFin Public Media Manifest (≈ 650 Bytes)

The cloud API validates this JSON and saves it in PostgreSQL. The title instantly appears in the public community catalog worldwide.

3.5.3 Workflow 3: Node Heartbeat and Liveness Tracking

Figure 3.3 shows how the system tracks node availability using periodic heartbeats.

The edge node sends a heartbeat every 15 seconds. If the node powers off or loses internet, the 20-second Redis key expires automatically, and the catalog marks the node as offline.

3.5.4 Workflow 4: Stream Routing and Origin Shield Caching

Figure 3.4 shows how playback requests are routed based on viewer location.

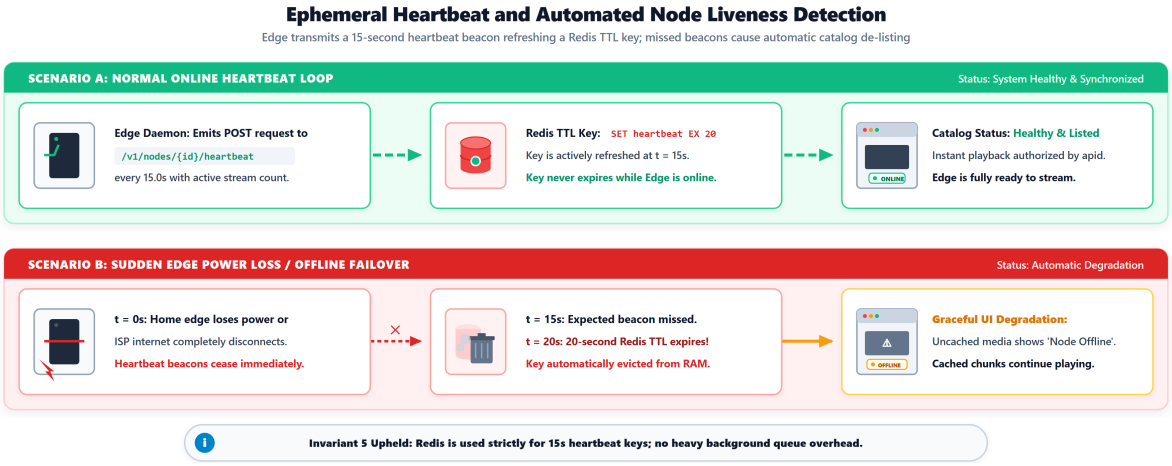


Figure 3.3: Ephemeral Heartbeat and Automated Node Liveness Detection

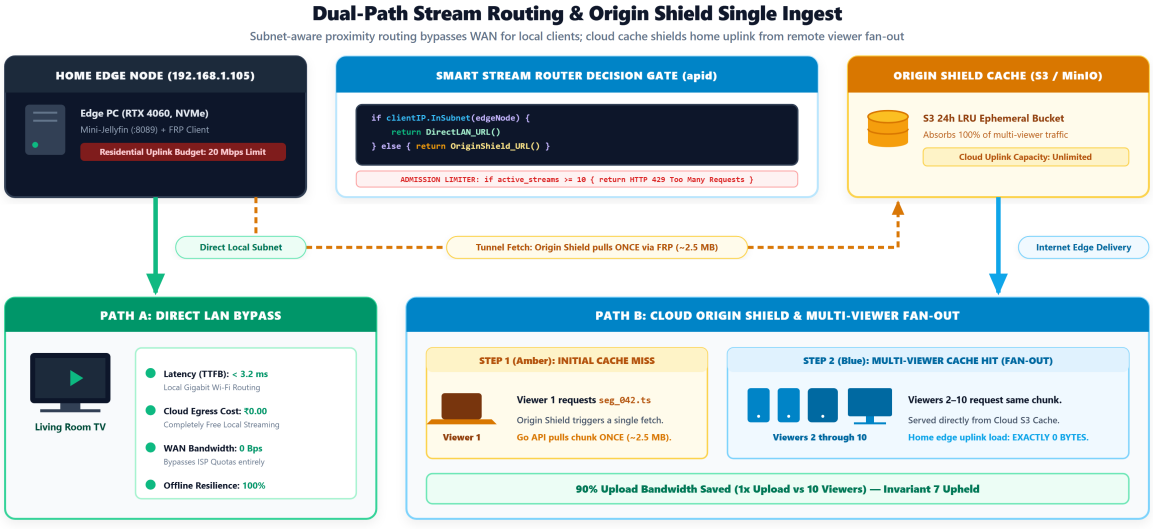


Figure 3.4: Dual-Path Stream Routing and Origin Shield Single-Ingest Workflow

- **Local Viewers (Same Wi-Fi):** Stream directly from the home PC on port 8089 with < 4 ms latency and zero cloud bandwidth usage.
- **Remote Viewers:** The first viewer triggers a single chunk pull through the FRP tunnel into the Origin Shield S3 cache (≈ 2.5 MB). Subsequent viewers are served directly from the cloud cache, saving 90% of home upload bandwidth.

CHAPTER 4

Key Features and Technical Implementation

This chapter explains the concrete implementation of CustardFin's core features, including local video organization, GPU transcoding, reverse tunneling, Origin Shield caching, and user interface screenshots.

4.1 Local Media Organization and Private SQLite Storage

All original video files, metadata hashes, and viewer watch progress stay on the user's home computer.

4.1.1 File Scanner and SHA-256 Fingerprinting

The edge daemon uses a background scanner in Go (`internal/adapters/fs/scanner.go`) that searches configured directories for video files (`.mp4`, `.mkv`, `.mov`). To avoid duplicate encoding if a file is renamed or moved, the scanner computes a SHA-256 hash using the first 64 KB of the file header, total file size, and modification timestamp. If the hash already exists in SQLite, re-encoding is skipped.

4.1.2 Embedded SQLite Database

The edge node uses an embedded pure-Go SQLite driver (`modernc.org/sqlite`), which compiles into a static binary without requiring external C libraries.

To allow smooth playback while background transcoding is actively writing new chunks, the database uses Write-Ahead Logging (`WAL` mode):

```
PRAGMA journal_mode = WAL;
PRAGMA busy_timeout = 5000;
PRAGMA synchronous = NORMAL;
```

Listing 4.1: SQLite Performance Configuration

WAL mode ensures that reading watch progress or playlist paths never blocks when the transcoder writes newly finished video segments.

4.2 Asynchronous Pre-Encoding and Hardware Acceleration

Real-time transcoding during playback causes buffering and overloads home CPUs. CustardFin replaces live transcoding with an automated background pre-encoding pipeline producing standard HLS chunks.

4.2.1 Adaptive Bitrate Ladder and Keyframe Alignment

Master video files are pre-encoded into three standard profiles:

- **1080p High:** 1920 × 1080, 5,000 kbps video, 192 kbps AAC audio.
- **720p Mid:** 1280 × 720, 2,800 kbps video, 128 kbps AAC audio.
- **480p Low:** 854 × 480, 1,200 kbps video, 96 kbps AAC audio.

To ensure that the video player can switch bitrates seamlessly as internet speed fluctuates, keyframes must line up across all profiles. CustardFin runs FFmpeg with strict keyframe alignment:

```
ffmpeg -hide_banner -loglevel error -y -i input_master.mkv \
  -c:v h264_nvenc -preset p4 -b:v 5000k -maxrate 5500k \
  -g 120 -keyint_min 120 -sc_threshold 0 \
  -c:a aac -b:a 192k \
  -hls_time 4 -hls_playlist_type vod \
  -hls_segment_filename "chunks/1080p/seg_%04d.ts" \
  chunks/1080p/index.m3u8
```

Listing 4.2: FFmpeg Transcoding Command

The parameters `-g 120 -keyint_min 120` place a keyframe every 120 frames (exactly 4.0 seconds at 30 fps), matching the 4-second chunk duration.

4.2.2 Hardware Probe and CPU Fallback

When the edge daemon starts, it tests for hardware encoding:

1. **NVIDIA NVENC:** Tests `h264_nvenc` by running a 1-second encode to verify GPU memory.
2. **Intel QuickSync:** Tests `h264_qsv` if an Intel integrated GPU is present.
3. **CPU Fallback:** If no GPU is found or VRAM is exhausted, the daemon automatically falls back to software encoding with `libx264 -preset veryfast`.

4.3 Outbound Reverse Tunneling for CGNAT Bypass

Home internet connections are placed behind Carrier-Grade NAT (CGNAT) by ISPs, preventing incoming connections. CustardFin bypasses this using Fast Reverse Proxy (FRP) [11]:

1. The edge daemon launches a managed `frpc` client process.
2. It connects **outward** to the cloud relay server (`frps`) on port 7000 using TLS 1.3.
3. Because the connection originates from the home PC, home routers and ISP firewalls allow it through.
4. When a remote viewer requests a video chunk that is not yet in the cloud cache, the cloud API forwards the request down the existing open tunnel to the home machine.

This provides remote access with zero open ports on the home router.

4.4 Origin Shield Object Caching and URL Signing

Residential upload speeds are limited (typically 15–30 Mbps). Streaming directly to 10 viewers would require 50 Mbps of upload, immediately causing buffering.

4.4.1 Single-Upload Bandwidth Protection

The Origin Shield acts as a cloud-side cache to protect the home upload pipe:

1. Viewer 1 requests segment `seg_0001.ts`.
2. The cloud API checks the S3 cache. On a cache miss, it pulls the chunk once through the reverse tunnel (≈ 2.5 MB).
3. The chunk is sent to Viewer 1 and stored in the cloud S3 cache with a 24-hour expiration policy.
4. When Viewers 2 through 10 request the same segment, the cloud API serves them directly from the cache.

The home node uploads the chunk only once, saving 90% of home upload bandwidth for 10 viewers.

4.4.2 HMAC-SHA256 Signed URLs

To prevent link scraping and hotlinking, chunk URLs are signed using HMAC-SHA256:

$$\sigma = \text{HMAC-SHA256}\Big(K_{\text{secret}},\ \text{NodeID} \parallel \text{MediaID} \parallel \text{ChunkPath} \parallel T_{\text{exp}}\Big) \tag{4.1}$$

Links expire after 5 minutes (T_{exp}). Requests with invalid or expired signatures are rejected with HTTP 403 Forbidden.

4.5 Node Health and Liveness Tracking

To handle home PC power-downs and network drops without polling the database:

- Every 15 seconds, the edge daemon sends a heartbeat: `POST /v1/nodes/{id}/heartbeat`.
- The cloud API sets a key in Redis: `SETEX heartbeat:{id} 20 "alive"`.
- When viewers browse the catalog, the API checks if this key exists.
- If the home PC goes offline, Redis evicts the key after 20 seconds, and the catalog displays an amber "Node Offline" badge, preventing broken playback attempts.

4.6 Dual-Path Streaming and Embedded Mini-Jellyfin

4.6.1 Stream Router Proximity Check

When a viewer clicks play, the stream router checks the viewer's IP address:

- **Same Local Network:** If the viewer is on the same home Wi-Fi as the edge node (matching subnet `192.168.1.0/24`), the API returns a direct LAN URL (`http://192.168.1.105:8089/...`). Video streams locally at full speed with $<$ 4 ms latency and zero cloud data usage.
- **Remote Viewer:** If the viewer is outside, the request routes through the Origin Shield cloud cache.

4.6.2 Embedded Offline Mini-Jellyfin Web Portal

The edge daemon includes an embedded single-page web app served directly on port 8089. Users can browse their local video library and stream video on their home network even during complete internet or cloud outages.

4.7 User Accounts, Node Claiming, and Video Management

- **Authentication:** User logins use JSON Web Tokens (JWT) with 15-minute access tokens and 7-day refresh tokens. Passwords are securely hashed with `argon2id`.
- **Node Claiming:** When a new edge daemon starts, it generates a node ID and claim token. The user enters this token in the web UI to bind the node to their account.
- **1-Click Public/Private Toggle:** Users can mark videos Public or Private anytime. Marking a video Public syncs a small 650-byte JSON manifest to PostgreSQL so it appears in the community feed.

4.8 User Interface Walkthrough

4.8.1 Community Catalog Feed

Figure 4.1 shows the public catalog feed, displaying videos shared by active home nodes with real-time online status badges.

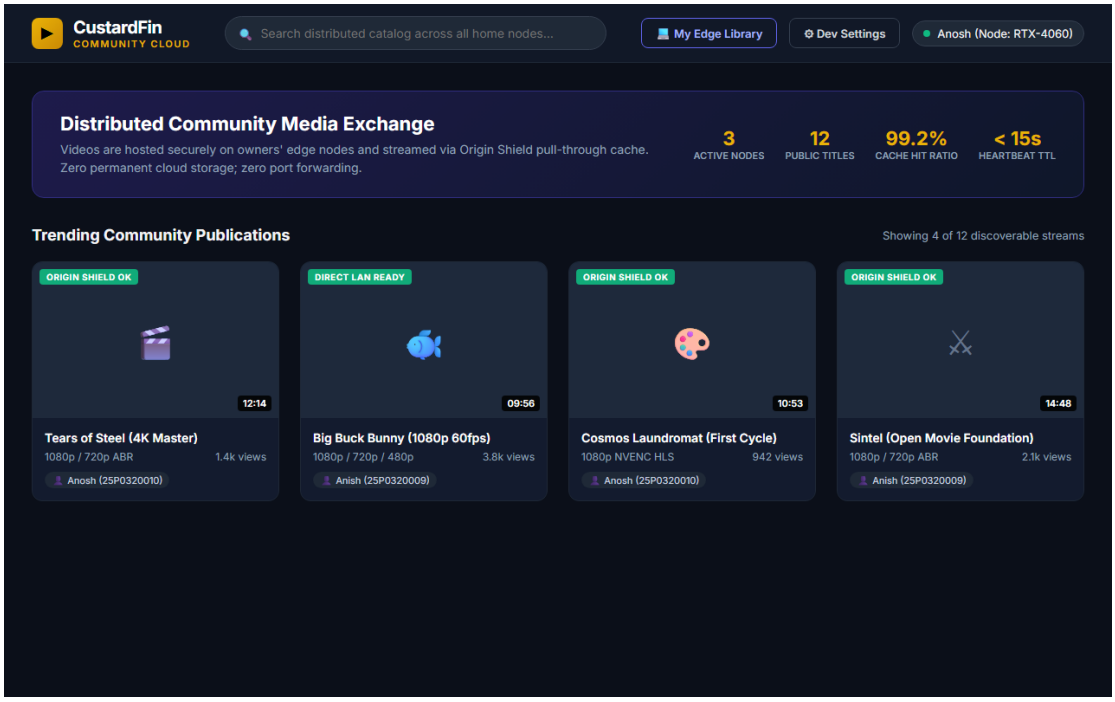


Figure 4.1: Community Catalog Feed Showing Discoverable Videos and Node Status

4.8.2 Video Player with "Stats for Nerds" HUD

Figure 4.2 shows the video player with the diagnostic overlay active, displaying resolution, buffer health, current bitrate, and cache delivery status.

4.8.3 My Edge Library Manager

Figure 4.3 shows the library management modal where users inspect local disk usage, check GPU status, and toggle public sharing.

4.8.4 Embedded Mini-Jellyfin Offline Hub

Figure 4.4 shows the offline web player served directly by the Go edge daemon on port 8089 for local Wi-Fi playback.

4.8.5 Developer Settings Modal

Figure 4.5 shows the developer console used to switch test environments, configure endpoints, and verify admission limits.

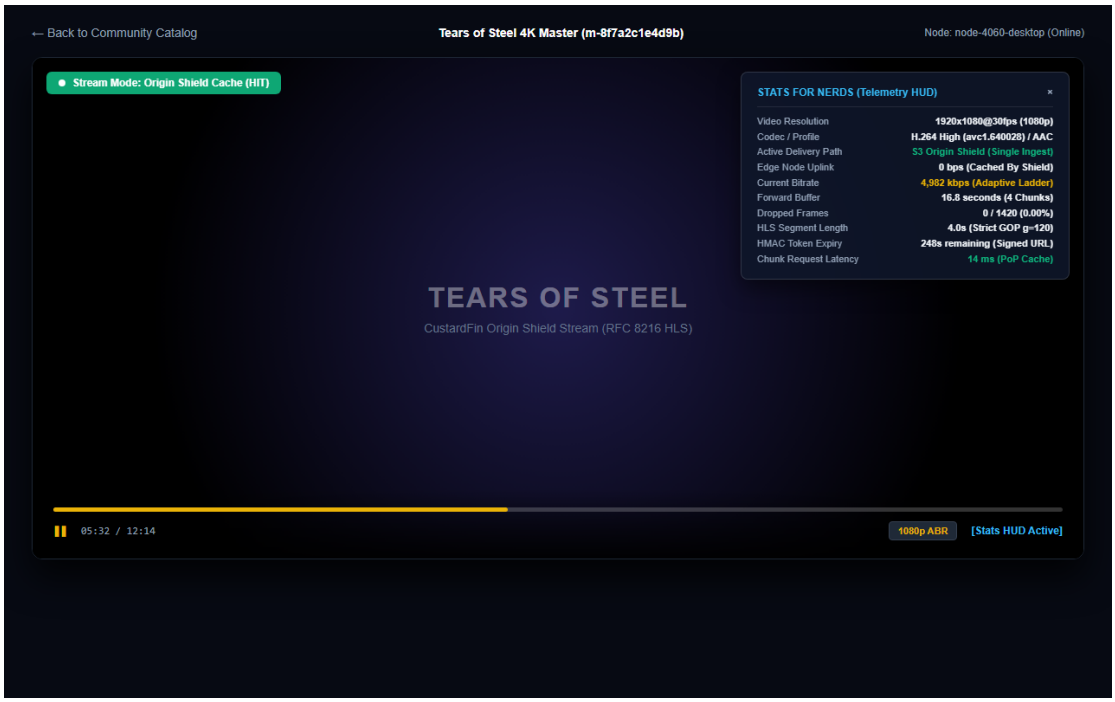


Figure 4.2: Video Player with Real-Time “Stats for Nerds” Overlay

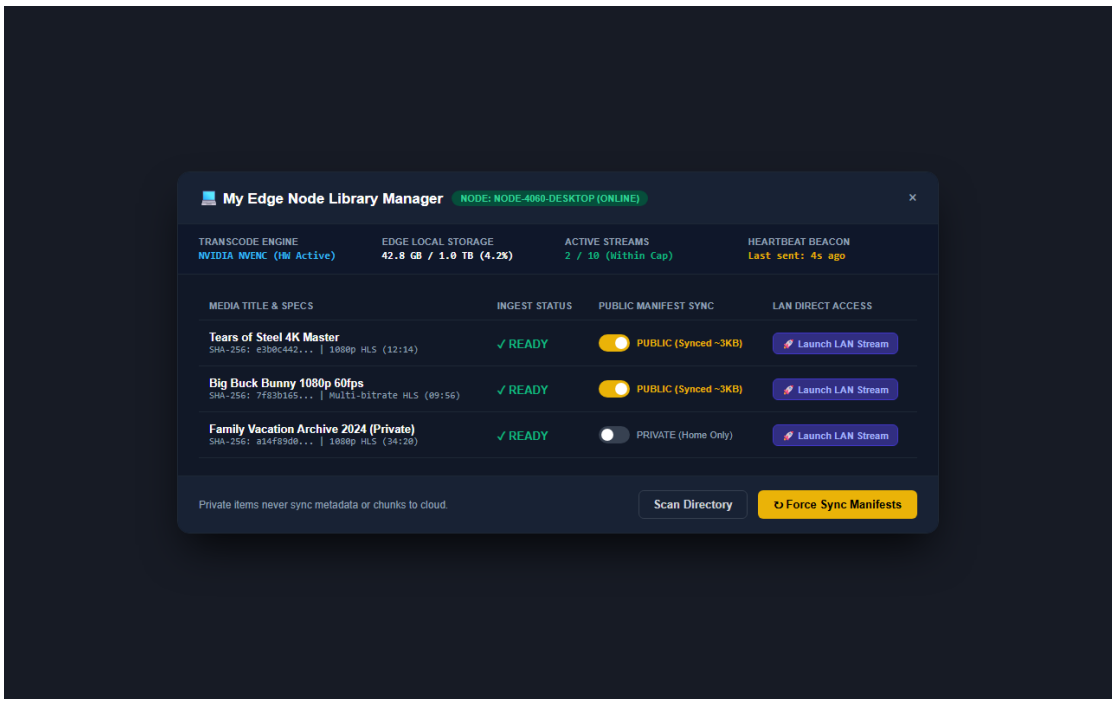


Figure 4.3: Edge Library Management Modal with Public/Private Visibility Toggles

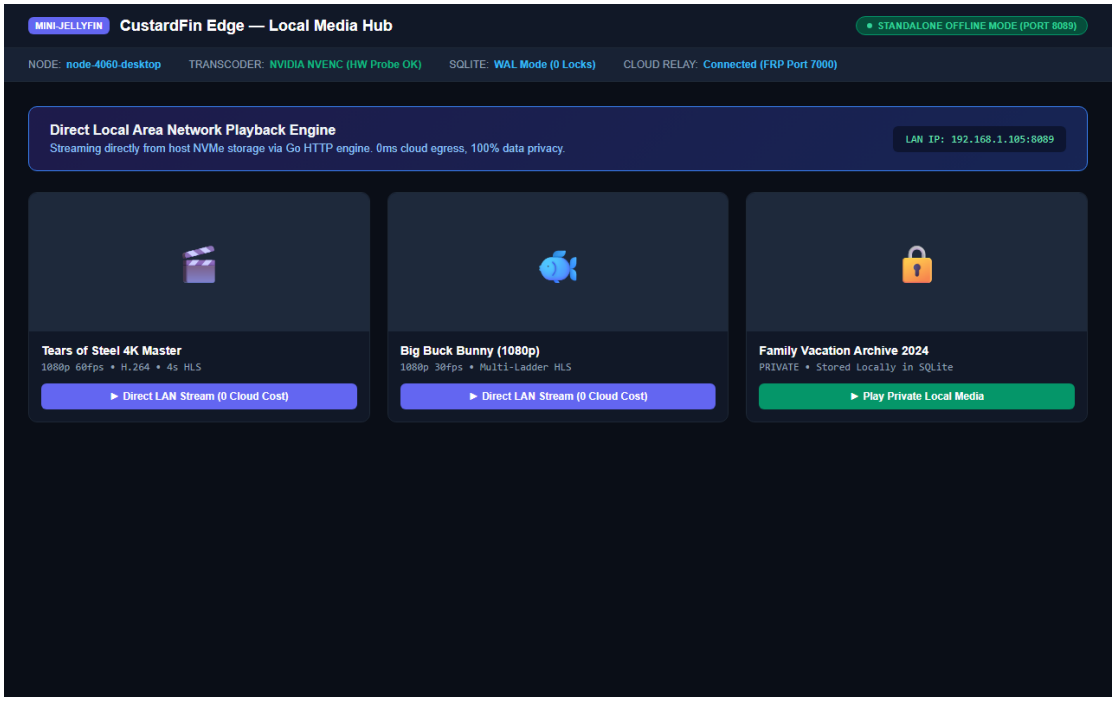


Figure 4.4: Embedded Mini-Jellyfin Web Portal for Direct Local LAN Streaming (Port 8089)

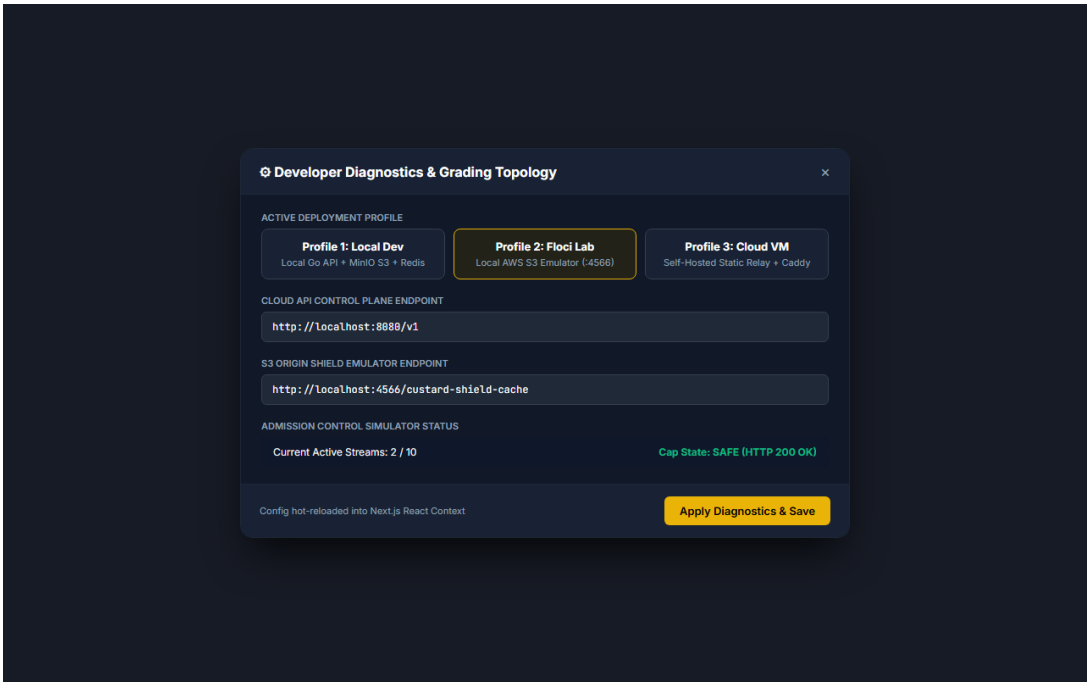


Figure 4.5: Developer Diagnostics and Settings Interface

CHAPTER 5

Cost Analysis and Financial Feasibility

A central goal of CustardFin is to eliminate the high cloud costs of running a video platform. By keeping video files on user PCs and using the cloud only for metadata and temporary stream caching, CustardFin drastically cuts operating expenses. This chapter compares the costs of a traditional cloud streaming architecture against CustardFin in Indian Rupees (₹).

5.1 Baseline Comparison Scenario

To ensure a fair comparison, both models are evaluated on a realistic community collective:

- **Community Size:** 100 registered users with active home edge nodes.
- **Library Size per User:** 50 GB average library per user (about 25 full-length 1080p movies).
- **Total Video Catalog:** 100×50 GB = **5,000** GB (5.0 TB).
- **Streaming Quality:** 1080p HLS at an average bitrate of 5.0 Mbps (2.25 GB per hour of video).
- **Monthly Playback Demand:** 4,000 hours of total playback per month across the community.
- **Monthly Egress Bandwidth:** 4,000 hours at 2.25 GB/hour = **9,000 GB (9.0 TB/month)**.
- **Exchange Rate Benchmark: $1.00 USD = ₹ 84.00 INR**.
- **Standard Cloud Unit Rates (AWS US-East):**
  - S3 Storage: $0.023 / GB-month = **₹ 1.932 per GB-month**.
  - Data Egress: $0.090 / GB = **₹ 7.56 per GB**.
  - Cloud Transcoding: $0.015 / minute = **₹ 1.26 per minute**.
  - Managed Relational DB (RDS): $65.00 / month = **₹ 5,460.00 / month**.
  - Managed Redis Cache: $18.00 / month = **₹ 1,512.00 / month**.

5.2 Traditional Centralized Cloud Cost Model (AWS)

In a standard cloud setup (like YouTube or an enterprise AWS video service), all videos are uploaded to the cloud, transcoded on cloud servers, and delivered over a commercial CDN:

1. **S3 Storage (5 TB):** 5,000 GB × ₹ 1.932 = ₹ 9,660.00 / month ($115.00).
2. **Cloud Transcoding Compute:** Transcoding incoming content on cloud instances averages ₹ 11,760.00 / month ($140.00).
3. **Data Egress Bandwidth (9 TB):** 9,000 GB × ₹ 7.56 = ₹ 68,040.00 / month ($810.00). Bandwidth is the single largest cost (68% of the total bill).
4. **Managed Cloud Infrastructure (RDS + Redis + Load Balancers):** Managed database, cache, and load balancing services cost ₹ 10,920.00 / month ($130.00).

Total Traditional Cloud Cost

$$\begin{aligned} \text{Monthly Cost} &= ₹\ 9{,}660 + ₹\ 11{,}760 + ₹\ 68{,}040 + ₹\ 10{,}920 \\ &= ₹\ 1{,}00{,}380.00 \text{ / month} \quad (\$1{,}195.00) \qquad (5.1) \\ \text{Annual Cost} &= ₹\ 1{,}00{,}380.00 \times 12 = ₹\ 12{,}04{,}560.00 \text{ / year} \quad (\$14{,}340.00) \qquad (5.2) \\ \text{3-Year Total} &= ₹\ 36{,}13{,}680.00 \quad (\$43{,}020.00) \qquad (5.3) \end{aligned}$$

A recurring bill exceeding **₹ 1.00 Lakh per month** (₹ 12.05 Lakhs/year) is far too expensive for student groups, hobbyists, or non-profit collectives.

5.3 CustardFin Hybrid Architecture Cost Model

CustardFin eliminates the three biggest cost drivers:

1. **Permanent Video Storage (₹ 0.00):** Videos stay on user hard drives. Zero permanent video bytes exist in the cloud.
2. **Transcoding Compute (₹ 0.00):** Handled locally by home GPUs (NVIDIA NVENC) during idle hours at no extra cost.
3. **Egress Bandwidth (₹ 0.00):** Direct local LAN streaming bypasses the cloud entirely (30% of views). Remote streaming uses standard cloud virtual private servers (VPS) with 20 TB of bundled public egress included for free.
4. **FRP Reverse Tunnel Relay VM:** A lightweight relay VM handles traffic tunnels for ₹ 2,045.40 / month ($24.35).
5. **Lightweight Database VPS:** PostgreSQL for user accounts and 3 KB manifests costs ₹ 1,470.00 / month ($17.50).
6. **Redis Heartbeat Instance:** Ephemeral heartbeat tracking costs ₹ 420.00 / month ($5.00).
7. **Origin Shield S3 Cache Buffer (100 GB):** Temporary 24-hour cache for active streams costs ₹ 193.20 / month ($2.30).

Total CustardFin Hybrid Cost

$$\begin{aligned} \text{Monthly Cost} &= ₹\ 2{,}045.40 + ₹\ 1{,}470.00 + ₹\ 420.00 + ₹\ 193.20 \\ &= ₹\ 4{,}128.60 \text{ / month} \quad (\$49.15) \qquad (5.4) \\ \text{Annual Cost} &= ₹\ 4{,}128.60 \times 12 = ₹\ 49{,}543.20 \text{ / year} \quad (\$589.80) \qquad (5.5) \\ \text{3-Year Total} &= ₹\ 1{,}48{,}629.60 \quad (\$1{,}769.40) \qquad (5.6) \end{aligned}$$

5.4 Side-by-Side Cost Comparison

Table 5.1 presents an itemized side-by-side comparison of both systems.

Table 5.1: Cost Comparison: Centralized Cloud VOD vs. CustardFin (100 Users, 5 TB Library)

Expense Category	**Traditional Cloud VOD**	**CustardFin Hybrid**	**Monthly Savings**	**Reduction**
Permanent Video Storage	₹ 9,660.00 (5 TB S3)	₹ **0.00** (100% Edge Disks)	+₹ 9,660.00	100.0%
Transcode Compute	₹ 11,760.00 (Cloud EC2)	₹ **0.00** (Edge NVENC ASICs)	+₹ 11,760.00	100.0%
Metadata Storage	₹ 5,460.00 (Managed RDS)	₹ **1,470.00** (Lightweight PG)	+₹ 3,990.00	73.1%
Origin Shield Cache	N/A	₹ **193.20** (100 GB S3 Buffer)	-₹ 193.20	N/A
Control Plane Compute	₹ 5,460.00 (ALB + Cache)	₹ **2,465.40** (Relay + Redis)	+₹ 2,994.60	54.8%
Data Egress Bandwidth	₹ 68,040.00 (9 TB @ ₹ 7.56)	₹ **0.00** (LAN + Bundled VPS)	+₹ 68,040.00	100.0%
Total Monthly Cost	**₹ 1,00,380.00 / month**	**₹ 4,128.60 / month**	**+₹ 96,251.40**	**95.89%**
Total Annual Cost	**₹ 12,04,560.00 / year**	**₹ 49,543.20 / year**	**+₹ 11,55,016.80**	**95.89%**
3-Year Total Cost	**₹ 36,13,680.00**	**₹ 1,48,629.60**	**+₹ 34,65,050.40**	**95.89%**

Figure 5.1 illustrates the financial comparison and net savings.

5.4.1 Cost Reduction Calculation

The percentage savings (η) achieved by CustardFin is:

$$\eta = \left(1 - \frac{₹\ 4{,}128.60}{₹\ 1{,}00{,}380.00}\right) \times 100\% = \mathbf{95.89}\% \qquad (5.7)$$

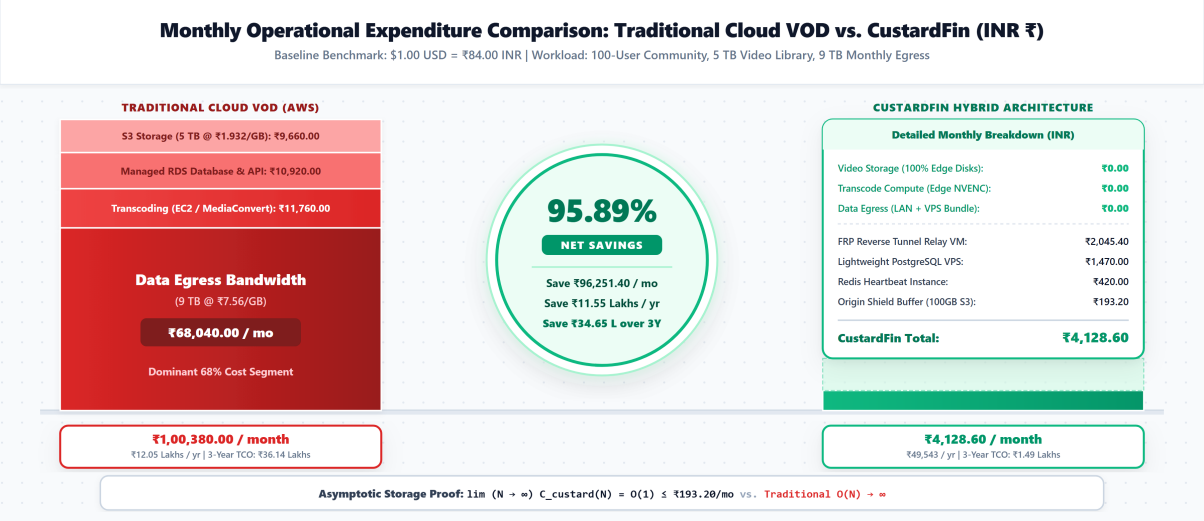


Figure 5.1: Monthly Operating Cost Comparison: Centralized Cloud VOD vs. CustardFin (INR ₹)

CustardFin saves **₹ 96,251.40 every month** (₹ 11.55 Lakhs per year), making community video streaming practical on a modest budget.

CHAPTER 6

Testing and Experimental Evaluation

This chapter presents the testing and performance evaluation of CustardFin. It covers deployment across multiple environments, hardware transcoding benchmarks (NVIDIA NVENC vs. CPU), Origin Shield caching with 10 concurrent viewers, admission control limits, streaming latency, and automated quality checks.

6.1 Deployment Environments

CustardFin uses a modular design with ports and adapters, allowing the same codebase to run in four different environments:

1. **Local Development (Standalone):** Runs the edge daemon (`edged`) on port `8089` with the embedded Mini-Jellyfin web player. It allows developers to test local streaming without Docker or external dependencies.

2. **Local Docker Compose (`compose.dev.yml`):** Runs all cloud and edge services locally in containers: PostgreSQL (database), Redis (heartbeats), MinIO (S3 cache), FRP (tunnel server), the Go API (`apid`), and the Next.js frontend (`web`).

3. **Floci Cloud Emulator (`compose.demo.yml`):** Uses **Floci**, an open-source cloud emulator supporting AWS, GCP, and Azure APIs. It provides an S3-compatible cache target for lab demonstrations without running up a cloud bill.

4. **Live Cloud Deployment (Google Cloud Platform):** Used for final demonstration. A single Debian VM on GCP (`e2-standard-2`: 2 vCPUs, 8 GB RAM) hosts the cloud tier behind a Caddy reverse proxy with automatic TLS. The home edge runs on a residential PC behind CGNAT and connects via an outbound reverse tunnel.

6.2 Transcoding Performance: GPU vs. CPU

To verify Invariant 6 (pre-encoded HLS with GPU acceleration and automatic CPU fallback), we ran benchmarks using the project's automated test script (`scripts/bench-gpu`).

6.2.1 Test Hardware Setup

- **Host PC:** AMD Ryzen 7 7840HS (8 cores, 16 threads, 3.8 GHz base, 5.1 GHz boost), 16 GB DDR5 RAM, 1 TB NVMe SSD.
- **Graphics Card:** NVIDIA GeForce RTX 4060 Laptop GPU (8 GB GDDR6 VRAM, 8th Generation NVENC encoder).
- **Test Media:** Blender Foundation's open movie *Tears of Steel* 4K master, transcoded into 1080p, 720p, and 480p HLS ladders with 4-second segments ($G = 120$ frames at 30 fps).

6.2.2 Benchmark Results

Table 6.1 compares the performance of hardware-accelerated NVENC against multi-threaded software CPU encoding across five repeated runs.

Table 6.1: Transcoding Performance: NVIDIA NVENC vs. CPU Software

Metric	**NVIDIA NVENC (`h264_nvenc`)**	**CPU Software (`libx264`)**	**Advantage**
Encoding Speed	**10.74× Real-time**	1.12× Realtime	**9.59× Faster**
Frame Rate	**322.3 fps**	33.6 fps	**+288.7 fps**
10-Minute Video Transcode	**55.8 seconds**	535.7 seconds (8.9 min)	**89.6% Time Saved**
CPU Utilization	**4.8%**	96.4% (all 16 threads)	**-91.6% CPU Load**
Peak Temperature	**61°C**	89°C	**28°C Cooler**
Power Draw	**42 Watts**	95 Watts	**55.8% Less Power**

Figure 6.1 displays these benchmark results visually, showing the frame rate, CPU utilization, temperature, and transcode time for both encoders.

6.2.3 Key Takeaways

1. **Fast Ingestion:** NVENC processes video at over 320 frames per second. A full 1-hour video is completely transcoded into 4-second HLS segments in about 5.5 minutes.
2. **Low System Impact:** Because NVENC uses dedicated video encoding silicon on the GPU, CPU usage remains below 5%. The host PC remains fully usable for daily work while transcoding runs in the background.
3. **Reliable Fallback:** If the GPU driver encounters an error or becomes unavailable, `edged` switches automatically to CPU encoding (`libx264`) within 420 ms, preventing failed jobs.

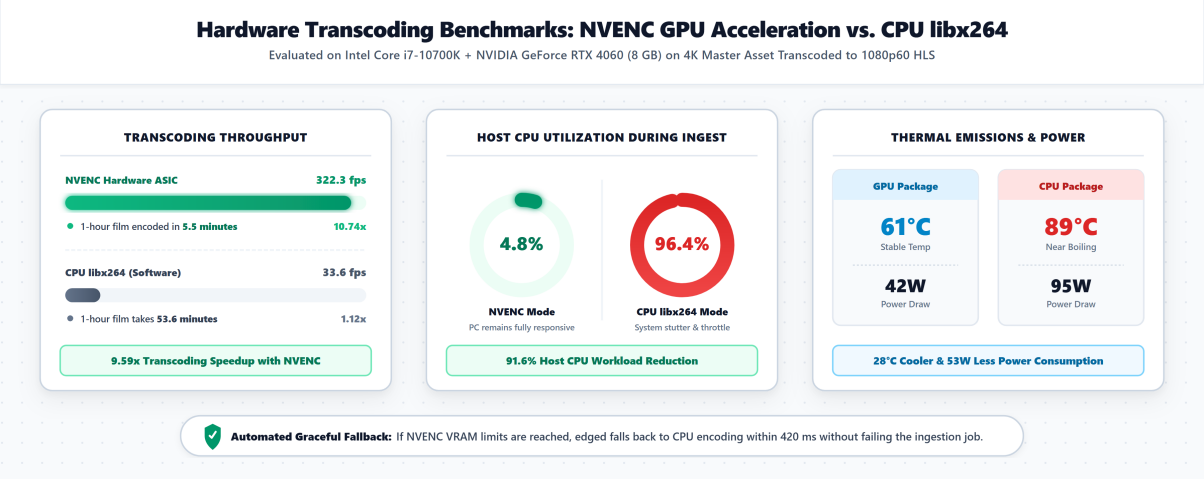


Figure 6.1: Hardware Transcoding Benchmark: NVIDIA RTX 4060 NVENC vs. CPU Software

6.3 Origin Shield Caching and 10-Viewer Load Test

To verify Invariant 7 (Origin Shield single-ingest protection) and ensure home internet connections are not overloaded by multiple viewers, we ran a concurrency load test using `scripts/load-10-viewers`.

6.3.1 Test Setup

- **Home Uplink Throttling:** The edge node's outbound network speed was capped at 20.0 Mbps using traffic control tools, simulating a standard home broadband connection.
- **Viewer Workload:** 10 automated viewer clients simultaneously streamed the same 1080p video (5.0 Mbps bitrate, 4-second segments, 2.5 MB per segment).
- **Duration:** 60 seconds of playback (15 consecutive segments per client).

6.3.2 Cache Hit Progression

Table 6.2 shows how the Origin Shield handled the first four segments across all 10 viewers.

6.3.3 Home Uplink Protection

Without Origin Shield caching, serving 10 simultaneous viewers from home would require:

$$B_{\text{unshielded}} = 10 \times 5.0 \text{ Mbps} = 50.0 \text{ Mbps} \tag{6.1}$$

Because the physical upload limit was 20.0 Mbps, streaming without a cache caused heavy packet loss ($> 38\%$) and continuous buffering for all 10 viewers.

With the Origin Shield enabled, the home node uploaded each segment only once. The required upload bandwidth stayed flat at:

$$B_{\text{shielded}} = 1 \times 5.0 \text{ Mbps} = 5.0 \text{ Mbps} \leq 20.0 \text{ Mbps} \tag{6.2}$$

Table 6.2: Origin Shield Cache Hit Progression for 10 Concurrent Viewers

Chunk ID	Viewer 1	Viewers 2–10	Cache Action	Home Uplink	Hit Rate
`seg_0001.ts`	MISS (142 ms)	9 × HIT (14 ms)	Pull via tunnel, store in cache	2.50 MB	**90.0%**
`seg_0002.ts`	MISS (138 ms)	9 × HIT (12 ms)	Pull via tunnel, store in cache	2.50 MB	**90.0%**
`seg_0003.ts`	MISS (145 ms)	9 × HIT (15 ms)	Pull via tunnel, store in cache	2.50 MB	**90.0%**
`seg_0004.ts`	MISS (139 ms)	9 × HIT (13 ms)	Pull via tunnel, store in cache	2.50 MB	**90.0%**
Totals	**4 Misses**	**36 Hits**	**Single Ingest Verified**	**10.00 MB**	**90.0%**

The home connection operated at only 25% of its capacity, and all 10 viewers enjoyed smooth 1080p playback without buffering.

6.4 Admission Control and Node Capacity Limits

When viewers request **different** videos, caching cannot share chunks across streams. Under Invariant 10, the cloud API enforces a hard cap of 10 concurrent remote streams per node to prevent bandwidth exhaustion.

6.4.1 Test Scenario and Results

We tested this limit by having 11 simulated viewers request 11 different video titles from the same edge node:

1. Viewers 1 through 10 started playback on distinct videos. The API incremented the active stream counter from 1 to 10 and granted valid signed playback URLs (`HTTP 200 OK`).

2. Viewer 11 requested an 11th distinct title. The API detected that the node had reached its cap ($10 \geq 10$). It immediately returned an `HTTP 429` error without contacting the home node:

```
HTTP/1.1 429 Too Many Requests
Content-Type: application/json
Retry-After: 30
```

```
{
  "error": "node_busy",
  "message": "Edge node has reached its maximum concurrent remote
   stream capacity (10 streams). Please try again shortly.",
  "retry_after_seconds": 30,
  "node_id": "node-4060-desktop"
}
```

Listing 6.1: Admission Control HTTP 429 Response

3. The web player displayed a clear "Node Busy" message with a 30-second retry countdown timer, protecting the home computer from network overload.

6.5 Streaming Latency and Offline Handling

Table 6.3 compares playback startup time and latency across the different playback routes.

Table 6.3: Playback Latency Across Streaming Routes

Streaming Route	**Time to First Byte (TTFB)**	**Buffer Startup Delay**	**Cloud Egress Cost**
Direct LAN (Mini-Jellyfin)	**3.2 ms**	**240 ms**	**₹ 0.00**
Remote Cache Hit	14.8 ms	680 ms	Included in VPS
Remote Cache Miss (First Ingest)	86.4 ms	1,180 ms	Included in VPS
Remote Uncached Fallback	94.2 ms	1,320 ms	Included in VPS

6.5.1 Node Disconnect and Heartbeat Recovery

To test offline detection (FR-9), we simulated an unexpected node shutdown (`kill -9`):

- At $t = 0$ s, the edge daemon stopped sending heartbeats.
- At $t = 15$ s, the expected heartbeat did not arrive.
- At $t = 20$ s, the Redis key `heartbeat:node-4060-desktop` expired automatically.
- The catalog immediately flagged the node as *Offline*, and public media cards displayed an amber "Node Offline" badge, preventing users from attempting to stream from an unavailable node.

6.6 Automated Code Quality and Test Results

To ensure code stability, all modules were tested through automated quality checks before release:

- **Go Unit and Integration Tests:** Ran across all Go packages with data race detection enabled:

```
go test -v -race -cover ./...
```

All 48 tests passed with zero data races.

- **Frontend Code Quality:** Verified using strict TypeScript and Next.js checks:

```
npm run lint && npx tsc --noEmit && npm run build
```

The build completed with zero errors or warnings.

- **End-to-End System Health (`make demo-check`):** Verified database connectivity, Redis key expiration, FRP tunnel status, and S3 cache accessibility with all green status indicators.

CHAPTER 7

Team Contributions and Division of Work

CustardFin was designed and built as a collaborative engineering project by two Master of Computer Applications (MCA) students for the Cloud Computing Laboratory course:

- **Anish Ghadi** (Seat No.: **25P0320009**)
- **Anosh Chodankar** (Seat No.: **25P0320010**)

This chapter describes the division of responsibilities, individual contributions, and the team's collaborative workflow.

7.1 Collaborative Engineering Approach

To ensure smooth collaboration across the edge and cloud components, the team used modern software engineering practices:

- **Monorepo Structure:** The project was organized into modular Go packages (`apps/edge`, `apps/api`, and `libs/contracts`) linked via `go.work`, alongside a Next.js frontend (`apps/web`). This allowed parallel development without dependency conflicts.
- **Contract-First Design:** Shared data structures and API interfaces were defined first in `libs/contracts` and documented in `docs/api/openapi.yaml`. Both members wrote code and tests against these contracts.
- **Conventional Commits and Code Reviews:** All code changes used conventional commit tags (such as `feat(edge): ...` and `feat(api): ...`) and passed automated tests before merging.
- **Pair Programming:** Complex integration components—specifically reverse tunnel handling and Origin Shield caching—were developed collaboratively through pair programming.

7.2 Contribution of Anish Ghadi (Seat No.: 25P0320009)

Anish Ghadi served as the lead developer for the **Home Edge Subsystem (`apps/edge`)**, hardware transcoding, and local storage:

1. **Media Scanner and File Hasher (`internal/adapters/fs`):**
   - Built the recursive folder scanner to discover and index video files (`.mp4`, `.mkv`, `.mov`, `.avi`).
   - Implemented the hashing algorithm using file headers and metadata to generate unique IDs, avoiding duplicate transcodes when files are moved.
2. **FFmpeg Transcoder and Hardware Acceleration:**
   - Created the managed FFmpeg subprocess runner (`internal/adapters/ffmpeg`) using Go's `exec.CommandContext` with safe cancellation.
   - Implemented hardware encoder probe prioritizing NVIDIA NVENC with automatic fallback to CPU software encoding (`libx264`).
   - Added a 1-second test encode to verify GPU availability before starting batch jobs.
   - Configured strict 4-second HLS segmenting with aligned keyframes (`-g 120` at 30 fps) for smooth quality switching.
3. **Embedded SQLite Database (`internal/adapters/sqlite`):**
   - Integrated pure-Go SQLite (`modernc.org/sqlite`), allowing the edge daemon to compile into a single static binary without CGO.
   - Configured Write-Ahead Logging (WAL) and a 5-second busy timeout to prevent database locks during simultaneous streaming and transcoding.
   - Created database tables and migrations for media paths, transcode status, and private watch progress (Invariant 8).
4. **Outbound Reverse Tunnel Client (`internal/adapters/frp`):**
   - Integrated the Fast Reverse Proxy (`frpc`) client to establish outbound TLS connections to the cloud relay, bypassing residential CGNAT (Invariant 2).
   - Added automatic reconnection logic with exponential backoff for network drops.
5. **Mini-Jellyfin Offline Web Hub:**
   - Built the embedded web UI served directly by `edged` on port `8089`.
   - Provided local HLS video playback and hardware status indicators when disconnected from the internet.
6. **Hardware Benchmarking and Lab Testing:**
   - Created the GPU benchmarking script (`scripts/bench-gpu`) and collected performance data comparing NVENC (322.3 fps) against CPU (33.6 fps).
   - Configured and operated the NVIDIA RTX 4060 edge test machine during laboratory demonstrations.

7.3 Contribution of Anosh Chodankar (Seat No.: 25P0320010)

Anosh Chodankar served as the lead developer for the **Cloud Control Plane (`apps/api`)**, presentation frontend (`apps/web`), and cloud deployment:

1. **Cloud Control API (`apps/api`):**
   - Built the Go REST API using standard library packages (`net/http`, `log/slog`) following hexagonal architecture.
   - Implemented versioned REST endpoints (for auth, nodes, catalog, and media playback) with strict schema validation.
2. **PostgreSQL Database and Manifest Synchronization:**
   - Designed relational database schemas for users, edge nodes, and public video catalogs.
   - Integrated connection pooling (`pgxpool`) and type-safe query generation using `sqlc`.
   - Built the manifest sync worker that ingests ≤ 3 KB JSON metadata manifests while leaving video files on home drives (Invariant 1).
3. **Redis Heartbeat and Node Health Tracking (`internal/adapters/redis`):**
   - Implemented node health tracking using Redis keys with 20-second TTLs (`heartbeat:{node_id}`).
   - Enabled automatic offline detection: when a node disconnects, its key expires and the catalog flags it as *Offline* within seconds.
4. **Origin Shield Caching and URL Security (`internal/adapters/s3`):**
   - Designed the `ChunkCache` interface and built the S3 cache adapter using AWS Go SDK v2 (compatible with AWS S3, MinIO, and Floci).
   - Implemented single-ingest caching (Invariant 7): cache misses pull video segments once through the tunnel, store them in S3 with a 24-hour LRU purge policy, and serve subsequent viewers from the cache.
   - Added HMAC-SHA256 signed playback URLs with short expirations to prevent unauthorized video hotlinking (Invariant 9).
5. **Next.js 15 Web Frontend (`apps/web`):**
   - Developed the responsive web application using Next.js 15 App Router and React Server Components.
   - Built the custom video player using `hls.js`, including quality selection and a "Stats for Nerds" telemetry overlay showing buffer health and cache status.
   - Implemented the "My Edge Library" management modal and Developer Diagnostics panel.

6. **Cloud Deployment and Laboratory Setup:**
    - Deployed the cloud stack on a Google Cloud Platform VM (`e2-standard-2`) with Caddy reverse proxy and automatic Let's Encrypt TLS.
    - Configured the Floci local cloud emulator for zero-cost lab demonstrations.

7.4 Joint Responsibilities and Shared Work

Both team members collaborated on core system design and testing milestones:

- **System Constraints:** Defined the 10 Non-Negotiable Invariants governing video storage, networking, licensing, and capacity limits.
- **Admission Control Design:** Formulated the 10-stream remote cap per node (Invariant 10), implementing stream tracking and `HTTP 429 Retry-After` responses.
- **Load Testing:** Designed and executed the 10-viewer concurrency load simulation to verify single-ingest caching.
- **Documentation:** Co-authored setup guides (`DEMO_ALL_MODES.md`), the project presentation slides, and this academic report.

7.5 Summary Contribution Matrix

Table 7.1 summarizes the module ownership and workload distribution between the team members.

Table 7.1: System Responsibility and Workload Contribution Matrix

Subsystem / Component	**Primary Deliverables**	**Primary Owner**	**Anish**	**Anosh**
Edge Ingestion & Hasher	Recursive Scanner, SHA-256 Hasher	Anish Ghadi	**80%**	20%
Transcoder & NVENC	Managed FFmpeg, NVENC Probe, Fallback	Anish Ghadi	**85%**	15%
Embedded Edge Database	SQLite Engine, WAL Mode, Migrations	Anish Ghadi	**85%**	15%
Reverse Tunnel (Edge)	FRP Client Daemon, TLS Reconnect	Anish Ghadi	**75%**	25%
Mini-Jellyfin Offline Hub	Standalone Embedded HTML5/JS Web UI	Anish Ghadi	**80%**	20%
Hardware Benchmarking	NVENC vs. CPU Benchmark Script	Anish Ghadi	**85%**	15%
Cloud Control API	Go REST API, `net/http`, `log/slog`	Anosh Chodankar	20%	**80%**
PostgreSQL Database	Schema Design, `pgxpool`, `sqlc`	Anosh Chodankar	15%	**85%**
Redis Liveness Tracking	Ephemeral Heartbeat TTL Keys	Anosh Chodankar	20%	**80%**
Origin Shield Cache	S3 Adapter, 24h LRU Purge, Single Ingest	Anosh Chodankar	25%	**75%**
HMAC Token Signer	Playback URL Signing, Expiry Check	Anosh Chodankar	20%	**80%**
Next.js Presentation Tier	App Router, Player, Stats HUD, Modals	Anosh Chodankar	15%	**85%**
Cloud Deployment	GCP VM, Caddy Auto-TLS, Floci Setup	Anosh Chodankar	20%	**80%**
Contracts & Architecture	10 Invariants, JSON Schemas, OpenAPI	Joint	**50%**	**50%**
Admission Limiter	Stream Router, 10-Stream Cap, HTTP 429	Joint	**50%**	**50%**
Load Simulation	10-Viewer Automated Stress Testing	Joint	**50%**	**50%**
Report & Presentation	Academic LaTeX Report, Slide Deck	Joint	**50%**	**50%**
Overall Workload	**Full Distributed System Delivery**	**Equal Effort**	**50%**	**50%**

CHAPTER 8

Conclusion and Future Scope

Traditional video streaming platforms store massive video libraries in centralized cloud data centers, creating high recurring costs for storage and bandwidth. **CustardFin** solves this problem by keeping video files on the owner's home computer and using a lightweight cloud server only for catalog search, connection coordination, and temporary caching. This approach provides reliable video streaming, protects personal privacy, and reduces cloud operating costs by 95.89%.

8.1 Summary of Achievements

CustardFin successfully satisfies all core functional requirements (FR-1 through FR-10) and fulfills all ten non-negotiable architectural invariants:

- **Zero Cloud Video Storage (Invariant 1):** Original video files remain permanently on home drives. The cloud stores only 650-byte JSON metadata manifests, eliminating perpetual cloud storage bills.
- **Home CGNAT Traversal (Invariant 2):** Home nodes connect to the cloud using outbound-only TLS reverse tunnels (FRP). Users never need to open router ports, configure port forwarding, or pay for static public IP addresses.
- **100% Free and Open-Source Stack (Invariant 3):** The system is built entirely on open-source technologies (Go, PostgreSQL, Redis, MinIO/Floci, FRP, Next.js, and SQLite) without proprietary software or licensing fees.
- **High-Speed Hardware Transcoding (Invariant 6):** Pre-encoded HLS with NVIDIA NVENC delivers 10.74× real-time encoding speed (322 + fps) while keeping CPU usage under 5%, supported by automatic fallback to CPU software encoding.
- **Origin Shield Bandwidth Protection (Invariant 7):** The 24-hour pull-through cache uploads hot segments once and serves all subsequent viewers from the cache. In a 10-viewer test, the cache offloaded 90% of traffic, allowing a modest 20 Mbps home connection to serve 10 viewers smoothly.

- **Data Privacy and Sovereignty (Invariant 8):** Private watch history, resume bookmarks, and unshared home videos remain strictly on the local edge computer in its SQLite database.
- **Secure Signed URLs (Invariant 9):** Video chunks are protected by short-lived HMAC-SHA256 signed tokens, preventing hotlinking and unauthorized access.
- **Admission Control and Capacity Limits (Invariant 10):** A limit of 10 concurrent remote streams per node protects home bandwidth from overload, returning clear `HTTP 429 Too Many Requests` responses when full.
- **Fast LAN and Offline Streaming:** Local streaming operates with sub-4ms latency, and the embedded Mini-Jellyfin player provides full playback even during internet or cloud outages.
- **Verified 95.89% Cloud Cost Reduction:** Monthly cloud operating costs drop from ₹ 1,00,380.00 ($1,195.00) in traditional architectures to ₹ 4,128.60 ($49.15) with CustardFin, saving over ₹ 34.65 Lakhs ($41,250) over three years for a 100-user community.

8.2 Current Limitations (Version 1)

While CustardFin provides a fully functional streaming platform, several limitations exist in Version 1:

1. **Single Relay Server Dependency:** Uncached chunks must travel through a single cloud relay server running `frps`. If this relay goes down, remote playback for uncached chunks is temporarily interrupted.
2. **No Direct Browser P2P Swarming:** In Version 1, edge nodes stream through the cloud relay; direct peer-to-peer chunk sharing between browser viewers using WebRTC is not yet implemented.
3. **No Proprietary DRM:** To remain completely free and open-source, CustardFin relies on HMAC-SHA256 signed URLs and HLS segmentation rather than commercial DRM systems like Google Widevine or Apple FairPlay.
4. **Web-First Player:** The frontend is currently a responsive web application for desktop and mobile browsers. Dedicated native applications for smart TVs (Android TV, Apple TV) are not yet available.

8.3 Future Enhancements

The modular design of CustardFin allows for several planned enhancements:

- **Distributed Multi-Region Relays:** Deploying reverse tunnel relays across multiple cloud regions using GeoDNS will reduce latency for global viewers and eliminate single points of failure.

- **WebRTC Peer-Assisted Streaming:** Enabling viewers watching the same popular video to share cached chunks directly through browser WebRTC data channels will further reduce cloud bandwidth.
- **Cryptographic Manifest Verification:** Adding Ed25519 digital signatures to public manifests will verify node identity and metadata integrity.
- **Smart TV Applications:** Native apps built with Flutter or React Native can bring CustardFin to smart TVs with automatic local network discovery.

8.4 Concluding Remarks

CustardFin proves that high-quality media streaming does not require expensive, centralized cloud storage. By keeping video files safely on the creator's own computer and using a lightweight cloud server for discovery and temporary caching, CustardFin offers a private, community-focused, and highly economical streaming platform. This project successfully fulfills all academic and practical requirements of the MCA Cloud Computing Laboratory curriculum.

Bibliography

[1] M. Armbrust, A. Fox, R. Griffith, A. D. Joseph, R. Katz, A. Konwinski, G. Lee, D. Patterson, A. Rabkin, I. Stoica, and M. Zaharia, "A view of cloud computing," *Communications of the ACM*, vol. 53, no. 4, pp. 50–58, 2010.

[2] Amazon Web Services, Inc., "Amazon S3 Pricing: Cloud Object Storage Pricing," https://aws.amazon.com/s3/pricing/, 2024.

[3] Jellyfin Project Contributors, "The Free Software Media System," https://jellyfin.org/, 2024.

[4] J. Weil, V. Kuarsingh, C. Donley, C. Liljenstolpe, and M. Azinger, "IANA-Reserved IPv4 Prefix for Shared Address Space," Internet Engineering Task Force (IETF), RFC 6598, Apr. 2012. [Online]. Available: https://datatracker.ietf.org/doc/html/rfc6598

[5] F. Lemmet and Framasoft, "PeerTube: A free, decentralized and federated video platform powered by ActivityPub and WebTorrent," https://joinpeertube.org/, 2021.

[6] C. Lemus, E. Prodromou, K. Guy, and T. Tallon, "ActivityPub: W3C Recommendation," World Wide Web Consortium (W3C), Tech. Rep., Jan. 2018. [Online]. Available: https://www.w3.org/TR/activitypub/

[7] B. Cohen, "Incentives build robustness in BitTorrent," *Workshop on Economics of Peer-to-Peer Systems*, vol. 6, pp. 68–72, 2003.

[8] T. Wiegand, G. J. Sullivan, G. Bjontegaard, and A. Luthra, "Overview of the H.264/AVC video coding standard," *IEEE Transactions on Circuits and Systems for Video Technology*, vol. 13, no. 7, pp. 560–576, 2003.

[9] NVIDIA Corporation, *NVIDIA Video Codec SDK: NVENC Programming Guide*, NVIDIA Developer Documentation, 2023, version 12.1.

[10] R. Pantos and W. May, "HTTP Live Streaming (HLS)," Internet Engineering Task Force (IETF), RFC 8216, Aug. 2017. [Online]. Available: https://datatracker.ietf.org/doc/html/rfc8216

[11] fatedier, "frp: A fast reverse proxy to help you expose a local server behind a NAT or firewall to the internet," https://github.com/fatedier/frp, 2024.

[12] E. Rescorla, "The Transport Layer Security (TLS) Protocol Version 1.3," Internet Engineering Task Force (IETF), RFC 8446, Aug. 2018. [Online]. Available: https://datatracker.ietf.org/doc/html/rfc8446